FOUNDING ENGINEER · TAKE-HOME ASSIGNMENT

Disaster Response Coordination Platform

Design & implementation document

Prepared for	Santrionn Technologies Private Limited
Repository	github.com/pavanreddy950/disaster-response-platform
Stack	Node.js · TypeScript · Express 5 · SQLite (R*Tree) · zod · Server-Sent Events · Vitest
Status	All core requirements + bonus (report priority) + operations dashboard · 70 automated tests passing

> **Approach in one line.** The brief asks to "*build less, think deeply, explain your decisions*". I kept the feature surface close to what was asked and spent the effort on failure handling, data integrity, edge cases and making every decision explainable. This document walks through what was built, why, and what I would do next.

Contents

1. Executive summary & requirement coverage
2. Architecture
3. Repository walkthrough (every file)
4. Data model
5. Key technical decisions & alternatives considered
6. Request flows
7. Edge cases handled
8. Testing strategy
9. Operations dashboard
10. Trade-offs, limitations & production roadmap
11. Running & demonstrating the project
12. AI-assisted development

1. Executive summary & requirement coverage

The platform is a backend API that lets responders **record disaster incidents**, **work out where they are** from free text, **find help nearby** (shelters, hospitals, food, water, rescue), **pull community reports** from an external social-media service, and **push live updates** to connected clients. Incidents follow an explicit **response lifecycle** and every change is recorded in a transactional **audit trail**. A thin operations dashboard sits on top of the public API to demonstrate it end to end.

Requirement (brief §)	How it is implemented	Where
1. Disaster CRUD + filtering, validation	REST endpoints; filters `tag`, `status`, `created_by`; pagination; strict zod schemas; uniform error envelope; soft delete	`modules/disasters/*`
2. Location resolution	Rule-based place extraction from the description, then a geocoder chain: cache → OpenStreetMap Nominatim (optional) → offline gazetteer fallback. Never blocks creation.	`integrations/geocoding/*`
3. Nearby resources	Two-phase geospatial search: SQLite R*Tree bounding-box index lookup, then exact haversine filter + sort. Antimeridian & pole handling.	`modules/resources/*`, `lib/geo.ts`
4. Community reports	HTTP client to a mock social API (real HTTP, latency, random failures, malformed items); normalisation; timeouts + retries	`integrations/social/*`, `modules/reports/*`
5. Caching	Read-through cache with TTL, request coalescing and stale-if-error. In-memory for reports (60 s), SQLite-persisted for geocoding (7 d / 1 h negative)	`lib/cache.ts`
6. Real-time	Server-Sent Events at `/events`: `disaster_created`, `disaster_updated`, `disaster_deleted`; in-process event bus	`realtime/*`
7. Auth & authorization	Bearer tokens → mock users (admin, contributor). Role gates + resource rules (ownership, admin-only reopen and delete)	`auth/*`, services
9. Testing	70 tests / 9 files: API, validation, business rules, integrations, cache, geo, SSE, migration	`tests/*`
10. README & docs	README with all requested sections; OpenAPI 3 (Swagger UI at `/docs.html`); Postman collection; `.env.example`; seed script	`README.md`, `docs/*`
Bonus	Priority classification of reports (high/medium/low); interactive map (dashboard)	`reports/priority.ts`, `public/*`
Beyond the brief	Incident lifecycle state machine (409 on invalid moves), transactional audit trail + history endpoint, versioned schema migration, request IDs, write rate limiting, `/stats`	see §5

Design principles I followed

Degrade, don't fail

A geocoder outage must not lose an incident report; a social-media outage should serve slightly stale data before it serves an error.

Rules live in one place

Lifecycle transitions are a data table, enforced by the service and published to clients, so the UI never duplicates business rules.

Database enforces invariants

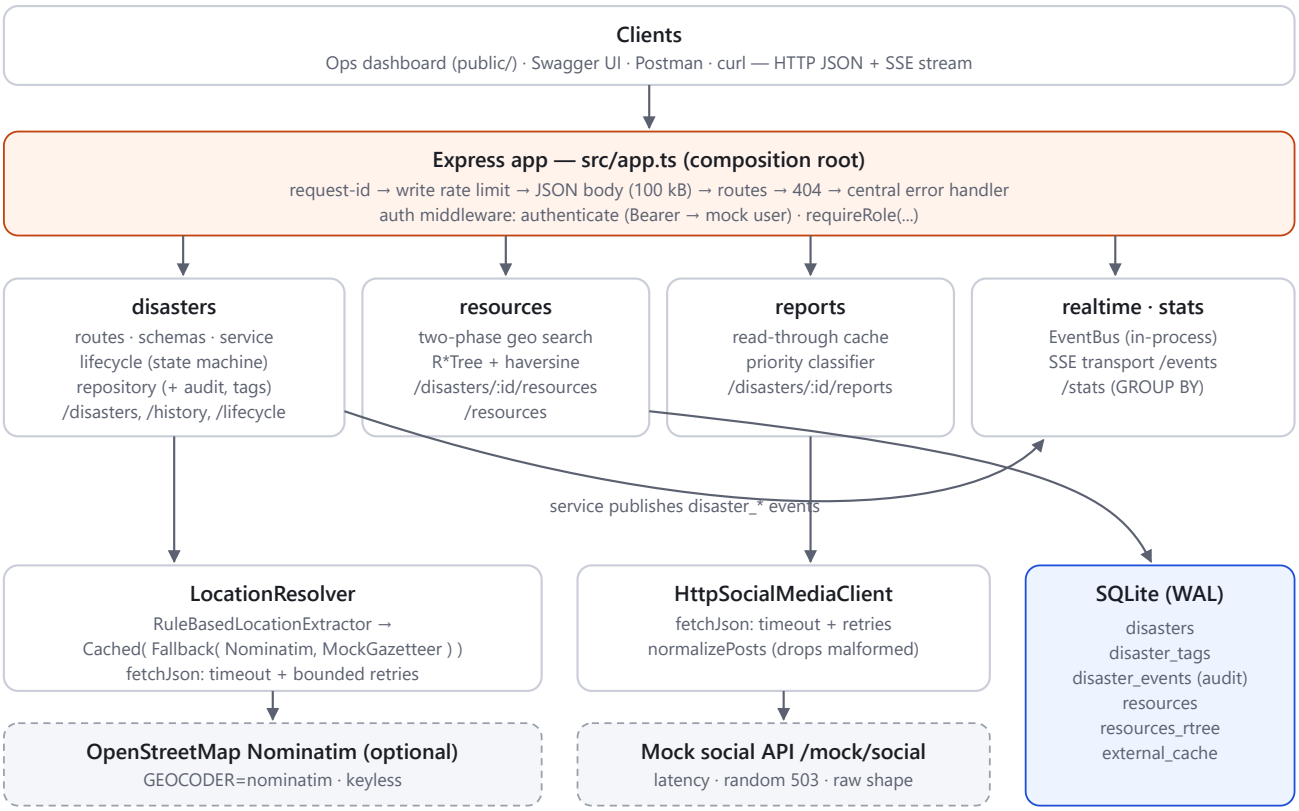
`CHECK` constraints, foreign keys and transactions back up the application-level validation.

Everything external is injectable

Geocoders, extractor and social client are interfaces passed into `createApp()`, so tests use fakes without module mocking.

2. Architecture

A single Node.js process with a layered, modular structure. Each feature module has **routes → service → repository**; integrations with the outside world sit behind small interfaces.



Layering rules

Layer	Responsibility	Not allowed to
Routes	Parse & validate input (zod), call one service method, shape the HTTP response (status, headers)	Contain business rules or SQL
Services	Business rules: ownership, lifecycle transitions, when to re-geocode, diffing for the audit trail, publishing events	Know about HTTP or SQL
Repositories	All SQL; transactions; row ↔ domain mapping	Make business decisions
Integrations	Talk to the outside world behind interfaces (`Geocoder` , `LocationExtractor` , `SocialMediaClient`)	Leak upstream data shapes into the app
Composition root	`createApp(deps)` wires everything; `server.ts` supplies real implementations, tests supply fakes	—

> Because every external dependency is injected, the same `createApp()` runs in production and in tests with an in-memory database and fake integrations. No global state, no module mocking.

3. Repository walkthrough

File	Purpose
Entry points & configuration	
`src/server.ts`	Process entry: loads config, opens DB, builds the real geocoder chain and social client, starts HTTP, graceful shutdown (closes SSE streams and DB)
`src/app.ts`	Composition root: wires repositories → services → routers, middleware order, static files, central error handler
`src/config.ts`	Validates environment variables with zod at startup (fail fast on bad config)
Database	
`src/db/schema.sql`	Idempotent DDL: tables, CHECK constraints, indexes, R*Tree virtual table + sync triggers, audit and cache tables
`src/db/connection.ts`	Opens SQLite (WAL, foreign keys, busy timeout), versioned migration via `PRAGMA user_version`
Shared library	
`src/lib/errors.ts`	`AppError` hierarchy (400/401/403/404/409/422/429/503) and `ExternalServiceError` with a `retryable` flag
`src/lib/validate.ts`	Turns zod issues into a field-level 400 response
`src/lib/cache.ts`	`CacheStore` interface, in-memory (bounded) and SQLite stores, `ReadThroughCache` (TTL, coalescing, stale-if-error)
`src/lib/http.ts`	`fetchJson`: per-attempt timeout, exponential backoff, retries only for retryable failures
`src/lib/geo.ts`	Haversine distance; bounding boxes for a radius incl. antimeridian split and pole handling
`src/lib/requestId.ts`, `rateLimit.ts`	Request correlation IDs; fixed-window write rate limiter with bounded memory
Auth & real-time	
`src/auth/users.ts`, `middleware.ts`	Mock token → user store; `authenticate`, `requireRole`, `currentUser`
`src/realtime/events.ts`, `sse.ts`	Typed in-process event bus; SSE endpoint with heartbeat, per-incident filter, cleanup on disconnect
Integrations	
`integrations/geocoding/extractor.ts`	Extracts a place name from free text (location cues, then gazetteer scan)
`integrations/geocoding/gazetteer.ts`	Offline place → coordinates table; resolves "Manhattan, NYC" by full string then comma parts
`integrations/geocoding/geocoders.ts`	`MockGeocoder`, `NominatimGeocoder`, `FallbackGeocoder`, `CachedGeocoder` (decorators)
`integrations/geocoding/locationResolver.ts`	Decides a disaster's location: client coords → name/extraction + geocode → unresolved

`integrations/social/socialClient.ts`	HTTP client + `normalizePosts` (upstream shape → `CommunityReport`)
`integrations/social/mockSocialService.ts`	Stand-in third-party API: raw shape, latency, configurable random 503s, one malformed post
Feature modules	
`modules/disasters/disaster.types.ts`	Domain types, status and audit-action enums
`modules/disasters/disaster.schemas.ts`	zod schemas for create / update / list query
`modules/disasters/lifecycle.ts`	Status state machine as data; `assertTransitionAllowed`; `GET /lifecycle` description
`modules/disasters/disaster.service.ts`	Create/update/delete rules, location re-resolution, field diff for audit, event publishing
`modules/disasters/disaster.repository.ts`	SQL: insert/update/soft-delete *with* audit row in one transaction; list with tag aggregation; history
`modules/disasters/disaster.routes.ts`	HTTP endpoints + authorization matrix
`modules/resources/*`	Resource schemas, R*Tree queries, nearby-search service, routes
`modules/reports/*`	Reports service (cache + error mapping), keyword priority classifier, routes
`modules/stats/*`	Counts per status and resource type for dashboard KPIs
Other	
`public/index.html, app.js, styles.css`	Operations dashboard (vanilla JS + Leaflet, no build step)
`docs/openapi.yaml`, `postman_collection.json`	API documentation; Swagger UI served at `/docs.html`
`scripts/seed.ts`	Idempotent demo data: 21 resources (NYC, Mumbai, Chennai, Kerala, Fiji), 5 incidents (one per status)
`tests/*.test.ts`	70 tests (see §8)

4. Data model

```
disasters                                    disaster_tags
──────────────────────────────────────────   ──────────────────────────────────────
id              TEXT  PK (UUID)        1──* disaster_id  FK → disasters (CASCADE)
title           TEXT  NOT NULL               tag          TEXT
description     TEXT  NOT NULL               PK (disaster_id, tag) · IDX (tag, disaster_id)
location_name   TEXT  NULL
lat, lng        REAL  NULL  CHECK range, CHECK both-or-neither
location_source TEXT  client | geocoded | unresolved          disaster_events  (audit trail)
status          TEXT  reported|verified|active|contained|resolved  ──────────────────────────
created_by      TEXT  (from authenticated user)          1──* id          INTEGER PK AUTOINCREMENT
created_at      TEXT  ISO-8601 UTC                            disaster_id FK → disasters
updated_at      TEXT                                          action      created|updated|status_changed|dele
deleted_at      TEXT  NULL  (soft delete)                     actor, changes (JSON diff), at
partial IDX (created_at) WHERE deleted_at IS NULL             IDX (disaster_id, id)

resources                                    resources_rtree  (R*Tree virtual table)
──────────────────────────────────────────   ──────────────────────────────────────
id            INTEGER PK AUTOINCREMENT 1──1 id
name          TEXT                          min_lat = max_lat = lat
type          shelter|hospital|food|water|rescue|medical|other    min_lng = max_lng = lng
location_name TEXT NULL                     kept in sync by INSERT / UPDATE / DELETE triggers
lat, lng      REAL  CHECK range
created_at    TEXT                          external_cache:  key PK · value JSON · expires_at
```

Modelling decisions

Decision	Reason
Tags in their own table	`?tag=flood` is an indexed lookup instead of a `LIKE` scan over a JSON/CSV column, and it is exact (no "flood" matching "flash-flood"). Tags are aggregated in the list query with `json_group_array`, avoiding N+1 queries.
`location_source` column	Makes provenance explicit: trusted client coordinates vs. best-effort geocoding vs. not yet known. Clients and operators can act on it.
Soft delete	Incident history matters for audit and post-incident review. Partial indexes on `deleted_at IS NULL` keep the live set fast.
Resources independent of disasters	A hospital exists regardless of any incident and can serve several. The disaster only supplies the default search centre.
UUID vs. integer ids	UUIDs for disasters (public, not enumerable); integers for resources because the R*Tree index requires integer keys.
Constraints in the database	Enums, coordinate ranges and both-or-neither coordinates are `CHECK` constraints, a last line of defence behind zod.
Append-only audit table	Monotonic ids give a total order even for equal timestamps; the JSON diff records exactly what changed.

5. Key technical decisions & alternatives considered

5.1 Database: SQLite with R*Tree

Chosen	SQLite via `better-sqlite3` (synchronous, very fast), WAL mode, foreign keys on.
Why	Zero setup for reviewers (`npm install` and run); a real spatial index (R*Tree) ships with SQLite; full relational features (FKs, CHECK, partial indexes, transactions); fast in-memory test databases.
Alternatives	PostgreSQL + PostGIS / Supabase: the production choice (`geography` column, GiST index, `ST_DWithin` + KNN ordering), but it requires a server or hosted account and credentials for every reviewer. MongoDB `2dsphere`: fine for geo, weaker for the relational parts (tags, audit, constraints).
Cost	Single writer, single node. All SQL is isolated in repositories, so moving to Postgres + PostGIS changes one layer.

5.2 Geospatial search: two-phase (index pre-filter + exact distance)

Phase 1	Compute a bounding box around the circle (Δlat = r / 111.32 km, Δlng = r / (111.32 km · cos lat)) and query the R*Tree: an index range search, roughly O(log n + k).
Phase 2	For the small candidate set, compute haversine great-circle distance, drop points outside the radius (the box corners), sort nearest first, apply `limit`; return `distance_km`.
Edge cases	Circle crossing ±180° is split into two boxes and merged; near the poles all longitudes are searched; radius bounded (0, 200 km]; R*Tree float32 rounding only affects the coarse box, the exact check uses the double-precision columns.
Alternative	Scanning all rows and computing haversine in SQL is simple but O(n); geohash prefixes are approximate at cell borders. PostGIS `ST_DWithin` is the production equivalent.

5.3 Location resolution: never block the incident

Order	1) client `lat/lng` → `client`; 2) `location_name` or place extracted from description, geocoded → `geocoded`; 3) otherwise `unresolved`.
Extraction	Deterministic rules: first capitalised phrase after a cue (in, near, affected, hit…), e.g. "affected Manhattan, NYC"; fallback scan for known place names. Behind a `LocationExtractor` interface so an LLM/NER extractor can replace it.
Geocoding	Decorator chain `Cached(Fallback(Nominatim, Gazetteer))`. Works fully offline by default; real provider optional via env.
Why	Recording a disaster matters more than having coordinates immediately. The state is explicit via `location.source`. On PATCH, location is re-resolved only when location-relevant fields change (saves geocoding calls).

5.4 Caching: read-through with coalescing and stale-if-error

Flow	fresh hit → return · miss/expired → one upstream call (concurrent callers share the same in-flight promise) → store → return · upstream fails → serve stale entry if any (`X-Cache: STALE`) → else 503.
Where	Reports: in-memory, bounded, 60 s TTL, keyed by upstream query (incidents in the same place share one call). Geocoding: SQLite-persisted, 7 days (1 h for negative results).
Why	Reports are time-sensitive; one minute absorbs bursts. Places don't move. Coalescing prevents a stampede when a hot key expires; stale-if-error keeps the product useful during upstream outages.
Scale-out	Move the store to Redis; the `CacheStore` interface is two methods.

5.5 External API failure handling

Transport	`fetchJson` : per-attempt timeout (3 s), exponential backoff, retries only for network errors, timeouts, 429 and 5xx. Never retries 4xx.
Normalisation	Upstream shape mapped to `{id, content, user, created_at, source}` ; malformed items dropped individually and logged; unexpected payloads yield an empty list instead of a crash.
Degradation	Geocoder: fallback gazetteer → `unresolved` . Social: stale cache → 503 `UPSTREAM_UNAVAILABLE` with a retry hint.
Realism	The mock social API is called over real HTTP and randomly fails 20% of the time, so failure paths run in normal use, not only in tests.

5.6 Real-time: Server-Sent Events

Chosen	SSE at `/events` , events `disaster_created / updated / deleted` with `{type, disaster, actor, at}` ; optional `?disaster_id=` filter; 25 s heartbeat; listener cleanup on disconnect.
Why	Updates are one-way (server → client). SSE is plain HTTP (proxy-friendly), needs no library, and browsers reconnect automatically.
Alternative	WebSockets / Socket.IO: bidirectional, which is not needed here, at the cost of extra infrastructure.
Scale-out	Services publish to an in-process `EventBus` ; back it with Redis pub/sub or Postgres `LISTEN/NOTIFY` so all instances fan out events.

5.7 Authentication & authorization

AuthN	`Authorization: Bearer <token>` → mock user store (admin, two contributors). Tokens are documented demo values, not secrets.
AuthZ	Two layers: route-level roles (`requireRole`) and service-level resource rules. Admin: everything. Contributor: create; update **only own** incidents; cannot delete or reopen resolved incidents. Reads are public. `created_by` always comes from the token (body field rejected).
Production	OIDC / signed JWTs from an identity provider; the rules stay the same.

5.8 Incident lifecycle (state machine)

Flow	`reported → verified → active ⇄ contained`; any open state → `resolved`; `resolved → active` admin only. New incidents start as reported, verified or active.
Errors	Invalid move → 409 `INVALID_STATUS_TRANSITION` with `details.allowed`. 409, not 400: the request is valid but conflicts with current state. Order: ownership 403 → reopen 403 → transition 409.
Why	Rules defined once as data, enforced by the service and published at `GET /lifecycle`, so the UI shows only valid actions without duplicating rules.

5.9 Audit trail & migration

What	Every create/update/status change/delete writes a `disaster_events` row with actor, timestamp and a field-level diff. Exposed at `GET /disasters/:id/history`.
Integrity	Written in the same transaction as the change, so an entry exists if and only if the change committed (tested by forcing the audit insert to fail). No-op PATCH writes nothing and emits no event.
Migration	Adding a status widens a CHECK constraint, which SQLite cannot alter in place. A versioned migration (`PRAGMA user_version`) rebuilds the table in a transaction with foreign keys temporarily off, then runs `foreign_key_check`. Tested against an old-schema database.

5.10 Validation, errors and API hardening

Validation	Strict zod schemas (unknown fields rejected), normalisation (trim, lower-case and dedupe tags), cross-field rules, bounded query params.
Errors	One envelope `{error:{code,message,details?,request_id}}`; unknown errors logged and returned as a generic 500 (no internals leaked); malformed JSON → 400; body > 100 kB → 413.
Hardening	Request IDs on every response, error and log line; per-user write rate limiting (429 + `Retry-After`), reads unlimited; config validated at startup.

6. Request flows

6.1 Create a disaster — `POST /disasters`

```
Client ── POST /disasters {title, description:"Heavy flooding has affected Manhattan, NYC."}
  │
  ├─ requestId → rateLimitWrites → express.json → authenticate → requireRole(admin|contributor)
  ├─ validate(createDisasterSchema)                    ── 400 VALIDATION_ERROR on failure
  ├─ DisasterService.create
  │    ├─ LocationResolver.resolve
  │    │    ├─ extractor: "Manhattan, NYC"
  │    │    └─ CachedGeocoder → (cache miss) → Fallback(Nominatim | Gazetteer) → 40.7831, -73.9712
  │    │       (any failure → location.source = "unresolved"; creation continues)
  │    ├─ repo.insert(disaster + tags + audit "created")  ── ONE transaction
  │    └─ EventBus.publish(disaster_created) ──► SSE clients
  └─ 201 Created, Location: /disasters/<id>, body = disaster
```

6.2 Nearby resources — `GET /disasters/:id/resources?radius=10`

```
load disaster (404 if missing/deleted) → validate query (400) → centre = query lat/lng ?? disaster coords (422
  → boundingBoxes(centre, r)  [1 box, or 2 across the antimeridian, or full-longitude near poles]
  → R*Tree join per box (indexed)  → merge/dedupe candidates
  → haversine distance → filter ≤ r → sort asc → limit → { center, radius_km, count, data[...distance_km] }
```

6.3 Community reports — `GET /disasters/:id/reports`

```
load disaster → query = location name ?? title → key "reports:<query>"
  → ReadThroughCache
       fresh?  ──► HIT (≈1 ms)
       in-flight? ──► await shared promise
       fetchJson(mock social, 3 s timeout, 2 retries w/ backoff) → normalizePosts → store (60 s) ──► MISS (≈40
       failed + stale entry ──► STALE        failed + nothing ──► 503 UPSTREAM_UNAVAILABLE
  → classifyPriority each report → sort high→low, newest first → optional ?priority filter
  → X-Cache header + { data, meta:{query, cache, fetched_at, count} }
```

6.4 Status change — `PATCH /disasters/:id {status}`

```
authenticate → validate → load → ownership (403) → reopen needs admin (403) → lifecycle table (409)
  → compute next state (re-geocode only if location-relevant fields changed)
  → diff(current, next): empty? → return 200 unchanged (no audit, no event)
  → repo.update(disaster + audit "status_changed" with diff)  ── ONE transaction
  → publish disaster_updated → 200
```

7. Edge cases handled

Area	Edge case → behaviour
Location	Geocoder down → incident saved as `unresolved` · no place in text → `unresolved` · lowercase place names → gazetteer scan · sentence boundary ("in Manhattan. Residents…") stops extraction · new description without a place keeps the old location · client-pinned coordinates are never overwritten by re-extraction
Geo search	Antimeridian (Fiji) · poles · points in the box corner but outside the circle excluded · unresolved disaster without query coords → 422 · lat without lng → 400 · radius ≤ 0 or > 200 km → 400
External APIs	Timeouts · 5xx/429 retried with backoff · 4xx not retried · invalid JSON · malformed items dropped individually · unexpected payload shape · thundering herd on cache expiry (coalesced) · outage with stale cache (served)
Validation	Unknown fields rejected (catches typos like `stauts`) · duplicate/mixed-case tags normalised · empty PATCH → 400 · malformed JSON → 400 · oversized body → 413 · unknown route → 404 envelope
Authorization	Spoofed `created_by` rejected · contributor editing another's incident → 403 · reopening resolved incident → admin only · delete → admin only (checked at route and service)
Data integrity	Audit row and change commit atomically · no-op updates produce no audit/event noise · soft-deleted incidents invisible to every read and to history · coordinates both-or-neither enforced in DB
Real-time	Disconnected clients unsubscribed (no listener leak) · heartbeats keep idle connections alive · graceful shutdown closes open streams · dashboard resyncs after reconnect
Abuse	Write rate limiting per user/IP with bounded memory · request IDs validated before being echoed into headers/logs · API strings never rendered as HTML in the dashboard

8. Testing strategy

70 tests across 9 files, ~2 s. Each test builds the real app via `createApp()` with an in-memory SQLite database and injected fakes for external services, then exercises it over HTTP with Supertest. The cache takes an injectable clock, so TTL behaviour is tested without sleeping; retry behaviour is tested against a real local HTTP server.

File	What it proves
`disasters.test.ts`	CRUD, filtering & pagination, location extraction + geocoding on create, client coordinates, geocoder outage still creates, field-level validation errors, strict schemas, malformed JSON, 401/403/404, ownership rules, soft delete, re-geocoding on description change
`lifecycle.test.ts`	Valid full path, 409 with allowed list on invalid moves, admin-only reopen, initial-status restriction, `/lifecycle` shape
`audit.test.ts`	History order, diffs and actors; no-op PATCH writes nothing; transaction rollback when the audit insert fails; 404 for unknown/deleted
`resources.test.ts`	Haversine accuracy, antimeridian split, poles, radius filter & ordering, box-corner exclusion, type filter, query validation, 422/404, admin-only create
`reports.test.ts`	Cache MISS → HIT with one upstream call, priority ordering & filter, 503 when upstream down, stale-if-error, TTL refresh, request coalescing, normalisation, priority classifier cases
`integrations.test.ts`	Location extraction cases, geocoder fallback, persistent geocode cache, `fetchJson` retries 5xx but not 4xx
`realtime.test.ts`	Opens a real SSE connection and asserts `disaster_created` and `disaster_updated` are delivered
`platform.test.ts`	Request-id echo/generation and presence in errors, 429 + `Retry-After` on writes only, `/stats` counts
`migration.test.ts`	Old-schema database migrates with rows, tags, indexes and foreign keys intact; second run is a no-op

> Coverage maps to the brief's minimum: one API endpoint ✓, one validation/error scenario ✓, one important business rule ✓, one external integration/mock ✓, each exceeded several times over.

9. Operations dashboard

The brief says a polished frontend is not required, so the dashboard is deliberately **thin**: one page of plain HTML/CSS/ES-module JavaScript, **no build step and no npm dependencies**, Leaflet for the map. It uses only the public API, which also demonstrates that the API is sufficient for a real client.



- **KPIs** from `/stats`: open incidents, per-status counts, unresolved locations, resources by type.
- **Map**: incidents coloured by status, resources by type; selecting an incident draws the search radius (slider) and highlights nearby resources.
- **Incident workspace**: overview with location provenance, lifecycle action buttons from `/lifecycle`, nearby resources with distances, community reports (priority filter, cache status, retry on 503), audit history timeline.
- **Role switcher** (Admin / Contributor 1 / Contributor 2 / Viewer): UI hides actions a role can't take but always surfaces the API's own 401/403/409 message, since the server is the authority.
- **Live updates** via `EventSource`: markers, list, KPIs and the open incident update in place; activity feed; resync after reconnect.
- Security: all API strings are rendered as text nodes (no HTML injection). Responsive to 375 px; light and dark themes.

10. Trade-offs, limitations & production roadmap

Intentional trade-offs

Decision	Gained	Gave up
SQLite + R*Tree over Postgres/PostGIS	Zero setup, real spatial index, fast tests	Single writer/node; manual bounding-box maths
Rule-based location extraction	Deterministic, free, offline, testable	Misses unusual phrasing (LLM/NER would be more robust)
In-memory reports cache & rate limiter	No infrastructure	Per-process; lost on restart; not shared across instances
SSE over WebSockets	Simplest correct tool for one-way push	No client → server messages (not needed)
Mock bearer tokens	Focus on authorization rules	No real identity, expiry or signing
Offset pagination	Simple	Can skip/duplicate rows under concurrent inserts
Synchronous geocoding in the create request	Coordinates in the 201 response	Request latency depends on the provider (bounded by timeout + retries)
Vanilla JS dashboard	Nothing to build or install	Less structure if the UI grows
Keyword priority classifier	Explainable, instant	Not semantic

Production roadmap (in priority order)

1. **Data & scale-out:** PostgreSQL + PostGIS (`geography` + GiST, `ST_DWithin`, KNN); Redis for cache, rate limiting and pub/sub behind the existing interfaces so SSE works across instances.
2. **Identity:** OIDC/JWT from an identity provider; per-organisation tenancy; rate-limit `/reports` as well (it fans out upstream).
3. **Asynchronous enrichment:** create incidents immediately, geocode and fetch reports in a background job queue, then emit `disaster_updated` (the real-time channel already exists).
4. **Concurrency control:** `ETag` / `If-Match` optimistic locking on PATCH to prevent lost updates.
5. **Data quality:** duplicate-incident detection (distance + time window + text similarity, operator confirms); LLM-assisted extraction and report triage with confidence and human override.
6. **Resources:** capacity and allocation with transactional over-allocation checks.
7. **Operations:** structured JSON logs, metrics (cache hit ratio, upstream latency/error rate), readiness probe, migration runner, cursor pagination.

11. Running & demonstrating the project

```
npm install
npm run seed      # data/app.db: 21 resources + 5 incidents (one per lifecycle status)
npm run dev       # http://localhost:3000
npm test          # 70 tests
npm run typecheck
```

URL	What
`http://localhost:3000/`	Operations dashboard
`http://localhost:3000/docs.html`	Swagger UI (OpenAPI spec in `docs/openapi.yaml`)
`http://localhost:3000/events`	Raw SSE stream (`curl -N`)
`docs/postman_collection.json`	Postman collection (create request stores `disasterId`)

Mock users

Token	User	Role
`admin-token`	admin	admin
`contributor-token`	contributor1	contributor
`contributor2-token`	contributor2	contributor

Five-minute demo script

1. Open the dashboard; note KPIs, map markers by status, the live indicator.
2. Create an incident as Contributor 1 with "Heavy flooding has affected Manhattan, NYC." Location is resolved, the marker appears and the live feed updates.
3. Select it: radius circle and nearby resources with distances; adjust the radius.
4. Open Reports: first load is `MISS`, reload is `HIT`; high-priority reports first.
5. Change status reported → verified → active; try an invalid jump via the API → 409 with allowed moves; the History tab shows each change with actor.
6. Switch to Contributor 2 and try to edit → 403; switch to Admin and delete → incident disappears live.

Configuration (`.env.example`)

PORT · DATABASE_PATH · GEOCODER (mock | nominatim) · NOMINATIM_URL · GEOCODER_USER_AGENT · SOCIAL_API_URL · SOCIAL_MOCK_FAILURE_RATE · SOCIAL_MOCK_LATENCY_MS · EXTERNAL_TIMEOUT_MS · EXTERNAL_RETRIES · REPORTS_CACHE_TTL_SECONDS · GEOCODE_CACHE_TTL_SECONDS · RATE_LIMIT_WRITES_PER_MINUTE. Every setting has a default; no API keys are required and none are committed.

12. AI-assisted development

Tool	Claude Code (Anthropic), a terminal-based coding agent
How it was used	As an implementation accelerator. It wrote most of the code, tests and documentation from my direction, and I used it to compare design options before choosing.
Decisions I made	**Database:** kept SQLite + R*Tree instead of switching to Supabase/PostgreSQL, so reviewers can run it with one command and the deadline wasn't at risk; the PostGIS path is documented. **Scope:** chose the extras that show engineering judgement (lifecycle, transactional audit trail, request IDs, rate limiting, thin dashboard), and rejected ones that only add surface area (Next.js rewrite, Redis/queues, Docker, LLM triage). **Delivery:** shipped a complete, tested baseline first, then iterated.

Extras I asked for beyond the brief

Request	Result
A frontend, so reviewers can experience the system without testing each endpoint by hand	Operations dashboard: map, incident workspace, live feed, role switcher
Make the system feel engineered rather than CRUD	Incident lifecycle (409 on invalid moves), transactional audit trail + `/history`, request IDs, write rate limiting, `/stats`
A document for reviewers explaining every choice	This design document
A video walkthrough of the real project	A 5-minute showcase built from real footage, code, test output and API responses (shared separately)
Plain-language explanations of the assignment and each approach	Used to make sure I can explain and defend every part of the design

How I validated it

Ran the full test suite (70 tests) and the type checker; exercised the API end to end (create, then nearby resources, then reports with cache MISS → HIT, then the 403 and 409 paths, then SSE events); ran the dashboard in a browser; reviewed the code module by module.

> I am comfortable walking through any part of this implementation, the reasoning behind each decision, and how I would evolve it for production.